

Amazon Reviews RAG Shopping Assistant: A Multi-Agent Project for Review-Grounded Product Recommendation

Chengzhe Li
UIUC
Champaign, USA
cli134@illinois.edu

Yongyang Pan
UIUC
Champaign, USA
yp21@illinois.edu

Zhicheng Tong
UIUC
Champaign, USA
zt19@illinois.edu

Xiaocheng Ma
UIUC
Champaign, USA
xm29@illinois.edu

Abstract

Online shoppers often need to compare many products whose useful information is scattered across long metadata fields and user reviews. Standard keyword search can miss products that match the intent of a natural-language shopping need, while large language models alone may produce recommendations that are not grounded in real product evidence. We developed an Amazon Reviews RAG Shopping Assistant, a FastAPI backend that ingests a sampled Amazon Reviews 2023 Electronics subset, builds product-level retrieval documents from metadata and review snippets, indexes them with vector search, and uses a multi-agent workflow to generate personalized, review-grounded recommendations. The system includes an offline ETL pipeline, Milvus-backed semantic catalog indexing, a three-agent recommendation runtime, a LangGraph pipeline variant, and an evaluation script comparing keyword retrieval with semantic retrieval. On a processed subset of 1,000 products and 10,000 reviews, semantic retrieval was often better at matching category-level intent, such as Bluetooth speakers, USB-C docks, and gaming headsets, while also revealing limitations from noisy product metadata, sparse review coverage, and fallback embeddings. The final artifact is a reproducible software prototype with documented setup, API usage, indexing commands, and evaluation scripts.

CCS Concepts

• **Information systems** → **Recommender systems**; *Retrieval models and ranking*; • **Computing methodologies** → *Natural language processing*.

Keywords

Retrieval-augmented generation, recommender systems, Amazon Reviews, semantic search, multi-agent systems, FastAPI, Milvus

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.
Conference acronym 'XX, Woodstock, NY

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM.
ACM ISBN 978-1-4503-XXXX-X/2018/06
<https://doi.org/XXXXXXXXXXXXXX>

ACM Reference Format:

Chengzhe Li, Zhicheng Tong, Yongyang Pan, and Xiaocheng Ma. 2018. Amazon Reviews RAG Shopping Assistant: A Multi-Agent Project for Review-Grounded Product Recommendation. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 5 pages. <https://doi.org/XXXXXXXXXXXXXX>

1 Introduction

E-commerce product search is still difficult when users express their needs in natural language. A query such as “portable bluetooth speaker with strong battery” contains intent, use case, and quality expectations. A keyword engine can match individual words, but it may rank products highly simply because they repeat common terms such as “portable” or “battery” without satisfying the whole shopping need. At the same time, a general-purpose large language model (LLM) can write fluent recommendations, but without retrieval it may hallucinate products or ignore real customer feedback.

Our project addresses this gap by building a review-grounded recommendation backend. The system uses the Amazon Reviews 2023 Electronics dataset [5] as the product evidence source, converts raw metadata and reviews into product-level documents, indexes those documents for semantic search, and then uses LLM agents to personalize and explain recommendations. The goal is not to implement a full commercial shopping site; instead, the project focuses on the backend intelligence layer that can understand natural-language shopping queries, retrieve relevant products, and summarize review evidence.

This project follows the development track. The main deliverable is the software system itself, with documentation that allows another user to download data, process a subset, build the vector index, call the recommendation API, and run the evaluation script. The source code repository for the project is:

<https://github.com/SheePPAN/E-commerce-Rec-Assistant>

2 Motivation and Intended Users

The target users are online shoppers and analysts who need to understand product options quickly. Shoppers benefit from concise product recommendations grounded in review highlights rather than isolated star ratings. Analysts benefit from a reusable pipeline

Table 1: Processed Amazon Reviews 2023 Electronics subset used by the prototype.

Artifact	Count
Products	1,000
Reviews	10,000
Product retrieval documents	1,000
Mock user profiles	50

for transforming large review datasets into searchable product documents.

The system is motivated by three common problems. First, keyword search does not reliably capture semantic shopping intent. Second, product reviews contain useful evidence, but they are too long and numerous for users to read manually. Third, LLM-generated recommendations need retrieval grounding to avoid generic or unsupported output. The developed tool combines retrieval and generation: retrieval narrows candidate products to items supported by dataset evidence, and generation converts the retrieved metadata and review highlights into concise recommendation text.

3 System Overview

The final system is a FastAPI [1] backend for an Amazon Electronics recommendation assistant. It contains two major stages: offline processing and online recommendation.

3.1 Offline Processing

The offline pipeline reads Amazon Reviews 2023 Electronics metadata and review files from local JSONL or JSONL.GZ files. It normalizes product records, normalizes review records, joins reviews to products using parent_asin, and writes processed artifacts to data/processed/. The processed subset used for the current prototype is shown in Table 1.

Generated files include products.jsonl, reviews.jsonl, product_documents.jsonl, mock_user_profiles.json, and etl_summary.json. Raw and processed data are ignored by git because the dataset is large.

3.2 Online Recommendation Runtime

At runtime, the user sends a recommendation request to the FastAPI API. The system then executes a three-agent workflow. UserProfileAgent infers a lightweight user profile from request context or feature-store data. ProductRecAgent recalls products using Milvus [3] semantic search, falls back to an English Amazon Electronics seed catalog when needed, and reranks candidates with an LLM. MarketingCopyAgent generates concise, review-grounded recommendation copy and a final recommendation summary. The runtime also includes a LangGraph [2] version of the same workflow to demonstrate graph-shaped orchestration.

4 Architecture

The architecture is organized around clear data and service boundaries:

Raw Amazon Reviews JSONL/GZ

-> Amazon ETL
 -> product_documents.jsonl
 -> EmbeddingService
 -> Milvus: amazon_electronics_products
 -> ProductRecAgent semantic recall
 -> LLM rerank
 -> TopN products
 -> MarketingCopyAgent copy and summary
 -> FastAPI response

Table 2 summarizes the major implementation components.

5 Implementation Details

5.1 ETL Pipeline

The ETL design streams the raw dataset instead of loading the full Electronics category into memory. This matters because the raw review file is several gigabytes. The prototype samples up to 1,000 products and 10,000 matching reviews.

```
selected_products = []
for record in metadata_file:
    product = normalize_product(record)
    if product matches target category:
        selected_products.append(product)
    stop when max_products reached
```

```
selected_ids = parent_asin values
reviews_by_product = {}
reviews_by_user = {}
```

```
for record in reviews_file:
    review = normalize_review(record)
    if review.parent_asin in selected_ids:
        append review to product and user maps
    stop when max_reviews reached
```

```
for product in selected_products:
    highlights = select review snippets
    source_text = combine metadata and highlights
    write product document
```

The resulting product document includes product title, category, brand/store, price, average rating, rating count, review count, selected features, description, review highlights, and source text for embedding.

5.2 Semantic Index

The vector index stores one embedding per product document. The default collection is amazon_electronics_products, and the embedding dimension is 384. The system uses Milvus for vector search and keeps a deterministic hash-based embedding fallback for offline tests and demos when model downloads or Milvus are unavailable. The production-intended embedding model is sentence-transformers/all-MiniLM-L6-v2 [4].

5.3 Recommendation Algorithm

The recommendation flow combines retrieval, personalization, and generation:

Table 2: Major software components and responsibilities.

Component	Responsibility
FastAPI app	Exposes health, recommendation, graph, catalog, and metrics endpoints
Amazon ETL	Streams raw metadata/reviews and builds product documents
AmazonCatalogIndex	Loads processed documents, embeds them, and indexes/searches Milvus
EmbeddingService	Uses all-MiniLM-L6-v2 embeddings with deterministic fallback
SupervisorOrchestrator	Runs profile, recall, rerank, copy, and summary workflow
LangGraph pipeline	Alternative graph representation of the same recommendation stages
MetricsCollector	Tracks in-memory agent success and latency statistics

```
input: user_id, optional query, context, num_items
```

```
profile = UserProfileAgent(context)
```

```
if query is provided:
```

```
    candidates, strategy = Milvus search(query)
```

```
    if no semantic results:
```

```
        candidates, strategy = fallback products
```

```
else:
```

```
    candidates, strategy = fallback products
```

```
ranked_ids = LLM rerank(query, profile, candidates)
```

```
top_products = preserve ranked order
```

```
copies = MarketingCopyAgent(top_products, profile)
```

```
summary = summarize(query, profile, top_products)
```

```
return products, copies, summary, metadata
```

The recall strategy is explicitly reported based on the actual recall path used. If Milvus search fails or returns no results, the response reports the fallback strategy rather than incorrectly labeling the result as semantic search.

5.4 API Endpoints

Table 3: FastAPI endpoints exposed by the backend.

Method	Path	Purpose
GET	/health	Health check
POST	/api/v1/recommend	Main recommendation endpoint
POST	/api/v1/recommend/graph	LangGraph endpoint
GET	/api/v1/catalog/status	Milvus catalog status
POST	/api/v1/catalog/reindex	Index processed documents
GET	/api/v1/metrics	In-memory agent metrics

6 How to Use the Software

6.1 Environment Setup

Install Python dependencies:

```
cd python
```

```
pip install -r requirements.txt
```

Download the Amazon Reviews 2023 Electronics review and metadata files from the UCSD dataset site [5] and place them locally:

```
data/raw/Electronics.jsonl.gz
```

```
data/raw/meta_Electronics.jsonl.gz
```

6.2 Run ETL

```
cd python
```

```
python -m services.amazon_etl \
```

```
--meta-path ../data/raw/meta_Electronics.jsonl.gz \
```

```
--reviews-path ../data/raw/Electronics.jsonl.gz \
```

```
--output-dir ../data/processed \
```

```
--category Electronics \
```

```
--max-products 1000 \
```

```
--max-reviews 10000
```

6.3 Start Milvus and API

```
docker-compose up -d
```

```
cd python
```

```
uvicorn main:app --host 0.0.0.0 --port 8000 --reload
```

6.4 Index Processed Documents

```
curl -X POST http://localhost:8000/api/v1/catalog/reindex \
```

```
-H "Content-Type: application/json" \
```

```
-d '{"processed_dir": "../data/processed",
```

```
"limit": 1000, "recreate": true}'
```

6.5 Request Recommendations

```
curl -X POST http://localhost:8000/api/v1/recommend \
```

```
-H "Content-Type: application/json" \
```

```
-d '{
```

```
  "user_id": "user_001",
```

```
  "query": "noise cancelling headphones for travel",
```

```
  "num_items": 5,
```

```
  "context": {
```

```
    "recent_views": ["headphones", "bluetooth speaker"],
```

```
    "avg_order_amount": 120
```

```
  }
```

```
}'
```

The response includes product records, generated recommendation copy, a summary grounded in product metadata and review highlights, and agent metadata such as recall strategy.

7 Evaluation

We implemented an evaluation script that compares keyword retrieval against semantic retrieval for curated shopping queries. The

Screenshot	placeholder.	Suggested	im-
age:	FastAPI	Swagger	page
/api/v1/recommend,	/api/v1/catalog/status,	and	showing
/api/v1/catalog/reindex.			

Figure 1: Backend API surface for the Amazon Reviews RAG assistant.

script can run locally with fallback embeddings or use Milvus when the vector index is available:

```
cd python
python evaluation/evaluate_retrieval.py \
--processed-dir ../data/processed
```

The current local evaluation used the processed 1,000-product subset and deterministic fallback embeddings. The results show that semantic retrieval often captures category intent better than raw keyword counting, while also exposing limitations in the small sampled corpus. Table 4 summarizes representative results.

The evaluation supports two conclusions. First, semantic retrieval is valuable for intent-oriented product discovery because it can retrieve category-relevant products even when exact keywords differ. Second, the current prototype is not a final production search engine: the small sample, noisy metadata, and fallback embeddings produce some mismatches. Running the full Milvus index with learned sentence-transformer embeddings should improve semantic ranking quality.

We also verified the implementation with automated tests:

```
python -m pytest
11 passed, 1 skipped
```

The skipped test is dependency-gated for optional API/runtime packages.

8 Sample Use Scenario and Screenshots

Figure 1 shows the intended demo workflow. In the final submitted PDF, this placeholder can be replaced by screenshots of the FastAPI Swagger page, catalog status response, recommendation JSON response, and evaluation output.

For a query such as “portable bluetooth speaker with strong battery,” the demo flow is: run ETL, reindex product documents into Milvus, submit a recommendation request, inspect returned products and review-grounded copy, and compare the result against keyword retrieval using the evaluation script.

9 Challenges and Design Decisions

9.1 Large Dataset Size

The raw Electronics review file is several gigabytes, so the ETL pipeline uses streaming and sampling instead of loading everything at once. This made development feasible while preserving the same data shape as the full dataset.

9.2 Product-Level Retrieval Documents

The raw dataset separates product metadata from reviews. We chose one document per product instead of one document per review because users typically ask for products, not individual reviews.

Product-level documents combine metadata and compact review highlights, giving the retriever enough evidence while keeping the index size manageable.

9.3 Retrieval Grounding

The LLM is used after retrieval rather than as the only recommendation source. This reduces hallucination risk and keeps generated explanations tied to product metadata and review highlights.

9.4 Fallback Behavior

Milvus may be unavailable in local demos, or a query may return no semantic hits. The system includes an English Amazon Electronics fallback catalog so the API remains usable. The response reports whether semantic recall or fallback recall was used.

9.5 Language Consistency

During development, some early Chinese e-commerce demo prompts and products were replaced with English Amazon Electronics prompts and fallback products. This avoids mixed-language instructions for an English dataset.

10 Limitations and Future Work

The prototype currently processes a small subset rather than the full Amazon Reviews 2023 Electronics category. Future versions should scale ingestion and indexing to more products, improve review aggregation into aspect-level pros and cons, and add stronger evaluation metrics such as human relevance judgments or precision at k.

The local evaluation used fallback embeddings unless Milvus and the sentence-transformer model are available. A more complete benchmark should compare keyword retrieval, fallback semantic retrieval, and Milvus sentence-transformer retrieval on the same labeled query set.

The current system is a backend prototype. A future frontend could provide interactive product comparison, filters, side-by-side review evidence, and user feedback collection.

11 Team Contributions

12 Conclusion

We developed a working Amazon Reviews RAG Shopping Assistant backend for the development track. The system transforms raw Amazon review data into searchable product documents, indexes products for semantic recall, orchestrates multiple agents for profile generation, reranking, and recommendation copy, and exposes reproducible API and evaluation workflows. The project demonstrates how retrieval-augmented generation can make product recommendation more grounded and useful than standalone keyword search or standalone LLM generation. The current prototype is limited by sample size and local embedding quality, but it establishes a clear software foundation for larger-scale review-grounded shopping assistance.

Acknowledgments

We thank the CS510 course staff for project guidance and the UCSD McAuley Lab for releasing the Amazon Reviews 2023 dataset.

Table 4: Representative qualitative comparison between keyword retrieval and local semantic retrieval.

Query	Keyword behavior	Semantic behavior
Portable bluetooth speaker with strong battery	Retrieved several products that matched isolated words, including DVD players and unrelated audio items.	Retrieved multiple Bluetooth speaker products, including waterproof and portable speakers.
USB-C hub for MacBook and monitor	Returned camera and backpack products because of incidental text matches.	Ranked a USB-C docking station and MacBook charger near the top.
Comfortable gaming headset with good microphone	Returned several unrelated products with weak keyword overlap.	Retrieved stereo headset, gaming headset, earbuds, and microphone-related candidates.
Compact camera for vlogging	Retrieved camera products with strong keyword matches.	Retrieved mini camera and digital camera-related products, but also some noisy results.
Durable phone case with screen protection	Mixed phone accessories with unrelated screen-protector mentions.	Retrieved screen protectors and protective cases, though not always phone-specific.

References

- [1] FastAPI. 2026. FastAPI Documentation. <https://fastapi.tiangolo.com/>. Accessed May 2026.
- [2] LangChain. 2026. LangGraph Documentation. <https://www.langchain.com/langgraph>. Accessed May 2026.
- [3] Milvus. 2026. Milvus Vector Database Documentation. <https://milvus.io/>. Accessed May 2026.
- [4] Sentence Transformers. 2026. Sentence Transformers Documentation. <https://www.sbert.net/>. Accessed May 2026.
- [5] UCSD McAuley Lab. 2023. Amazon Reviews 2023 Dataset. <https://amazon-reviews-2023.github.io/main.html>. Accessed May 2026.